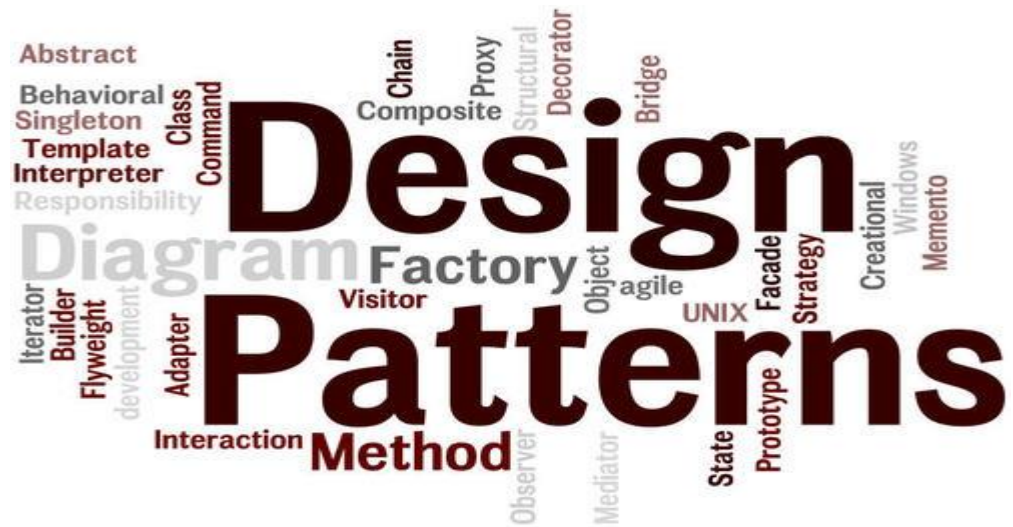




Mẫu Singleton

ThS. Trần Anh Dũng
dungta@uit.edu.vn

Nội dung

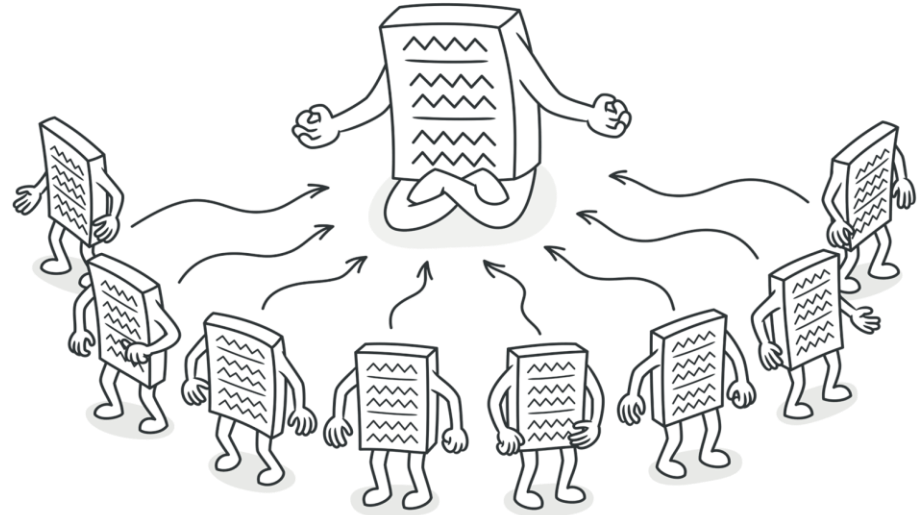
- Tổng quan
 - Tên
 - Phân loại
 - Mục đích, ý nghĩa
- Motivation
- Khả năng ứng dụng
- Đặc điểm:
 - Cấu trúc mẫu
 - Ý nghĩa của từng thành viên
 - Sự cộng tác

Nội dung

- Các hệ quả mang lại:
 - Ưu điểm
 - Nhược điểm
- Các chú ý liên quan đến cài đặt, demo, mã nguồn minh họa
- Một số so sánh nếu có
- Nêu ra các ví dụ thực tế
- Các mẫu liên quan

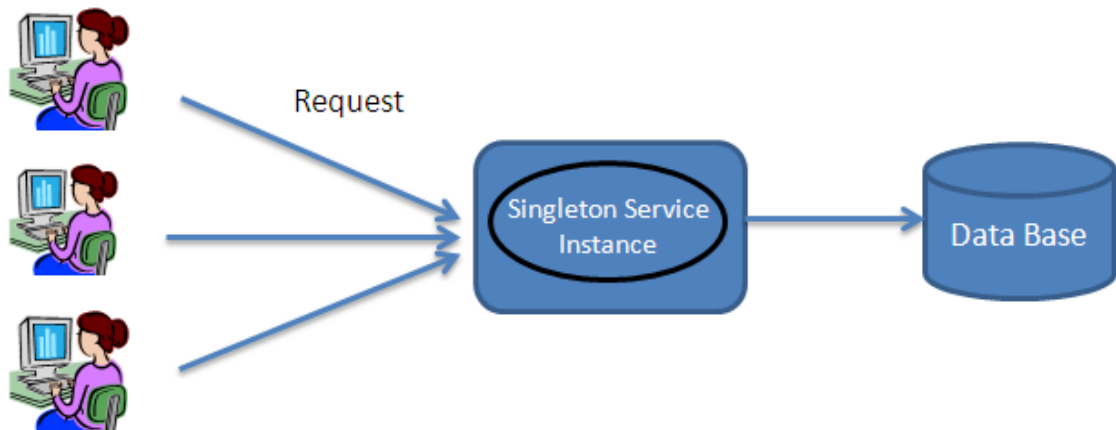
Tổng quan

- Tên mẫu:
 - Singleton
- Phân loại:
 - Thuộc phân nhóm **Creational patterns**
- Mục đích:
 - Kiểm soát việc tạo lập đối tượng.
 - Luôn luôn **chỉ có 1 instance duy nhất cho Class**.
Instance này phải được truy xuất ở phạm vi **Global**
 - Luôn luôn sẵn sàng để được sử dụng

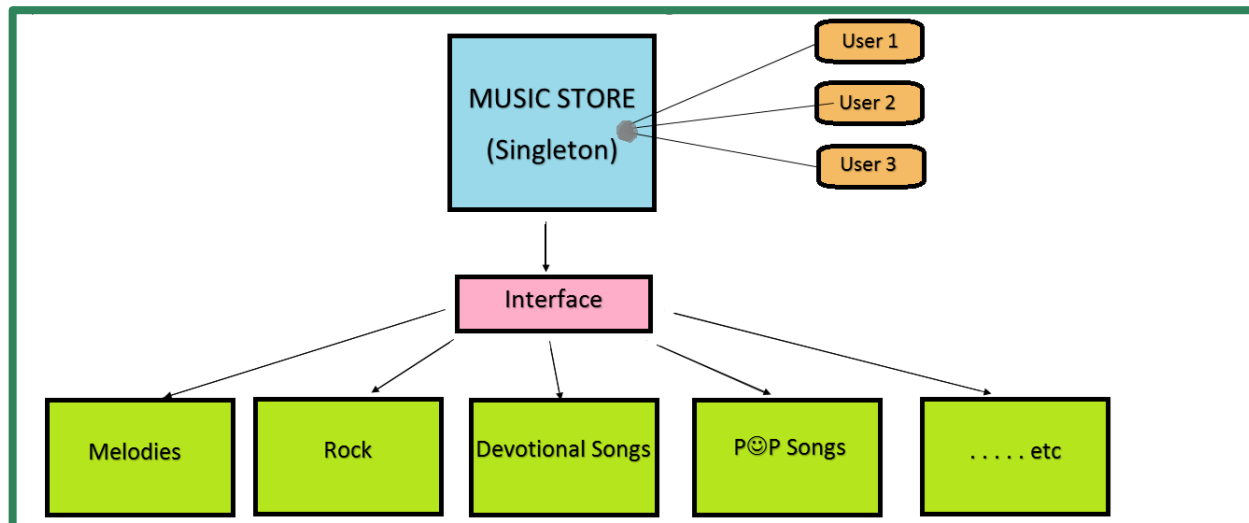
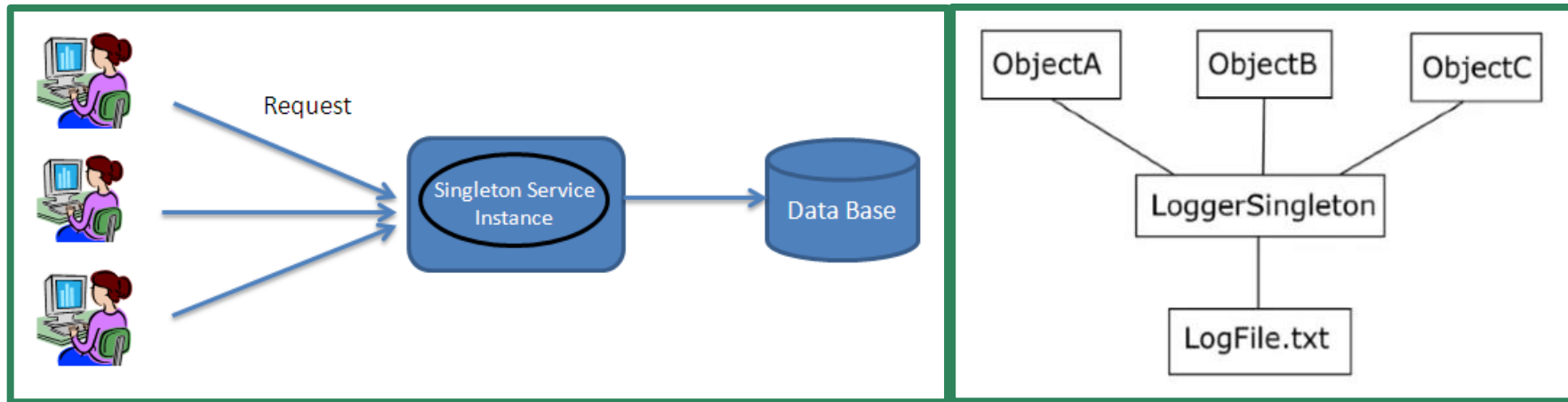


Motivation

- Cần duy nhất một instance của một class nào đó trong system.
- Cần truy cập, sử dụng instance này dễ dàng.
- Ví dụ: window manager, Audio device, Printer spooler, những tài nguyên dùng chung (DB, file,...) chỉ cần dùng 1 instance mỗi lần được request.

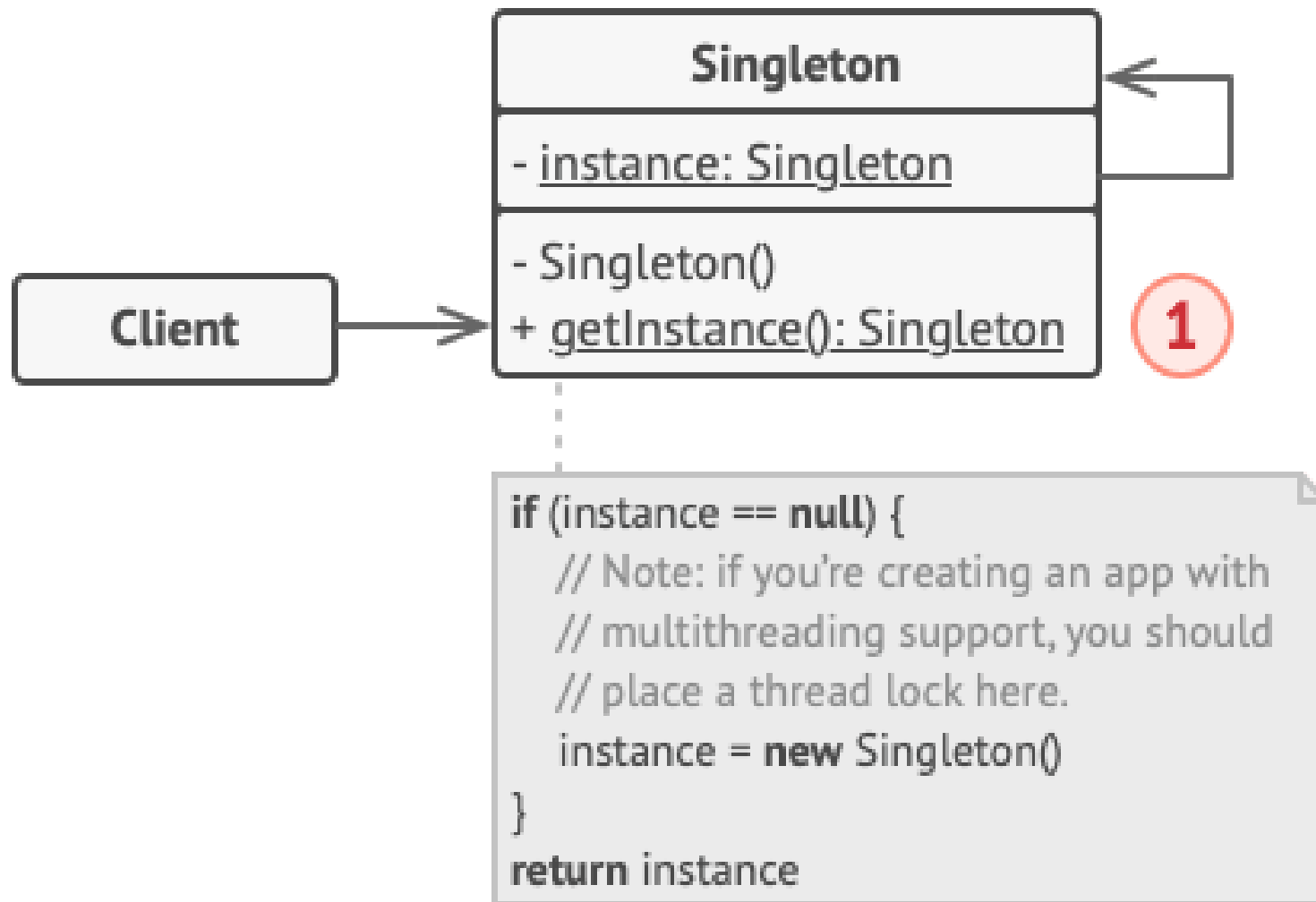


Khả năng ứng dụng



Đặc điểm

- Cấu trúc mẫu:

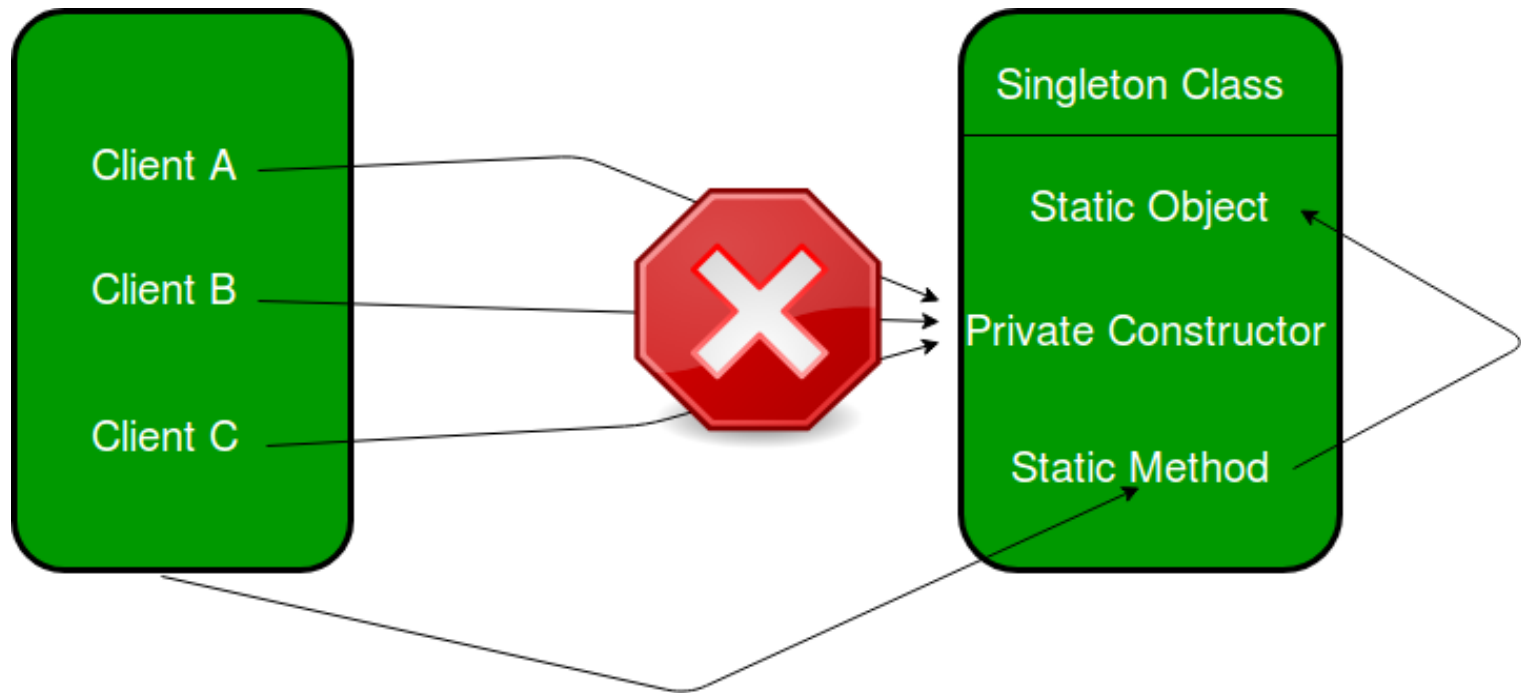


Đặc điểm

- Ý nghĩa của từng thành viên:
 - **Dữ liệu thành viên instance** (private và static) là đối tượng duy nhất của lớp Singleton.
 - **Constructor** của lớp Singleton được định nghĩa thành protected hoặc private.
 - **Phương thức getInstance()** dùng để khởi tạo đối tượng duy nhất, định nghĩa thành public và static. Client chỉ dùng getInstance() để tạo đối tượng cho lớp Singleton.

Đặc điểm

- Sự cộng tác:



- Clients truy cập, **lấy instance** của Singleton Class này không phải thông qua Constructor mà phải thông qua một **Static Method**.

Hệ quả mang lại

- Ưu điểm:
 - Đảm bảo truy cập vào duy nhất một instance.
 - Đối tượng singleton chỉ được khởi tạo duy nhất một lần khi nó được yêu cầu lần đầu.
 - Linh hoạt hơn một class bình thường.
 - ...

Hệ quả mang lại

- Nhược điểm:
 - Pattern này yêu cầu được đối xử đặc biệt trong môi trường multithreaded để nhiều threads không thể tạo một đối tượng singleton nhiều lần.
 - Gây khó khăn trong kiểm thử
 - ...

Hiện thực/cài đặt

- Eager initialization
 - Singleton Class được khởi tạo ngay khi được gọi đến.
- Static block initialization
 - Static block cung cấp thêm lựa chọn cho việc handle exception hay các xử lý khác.
- Lazy Initialization
 - Là một cách làm mang tính mở rộng hơn so với 2 cách làm trên và hoạt động tốt trong môi trường đơn luồng (single-thread).

Hiện thực/cài đặt

- Thread Safe Singleton
 - Phương thức synchronized của hàm getInstance()
- Double Check Locking Singleton
 - Khai báo volatile cho instance để tránh lỗi làm việc không chính xác do quá trình tối ưu hóa của trình biên dịch
- Bill Pugh Singleton
 - Tạo ra static nested class với vai trò 1 Helper khi muốn tách biệt chức năng cho 1 class function rõ ràng hơn.

Hiện thực/cài đặt

- Eager initialization

- Đây là cách khởi tạo một lớp Singleton dễ và đơn giản nhất, theo cách này thì thể hiện sẽ được khởi tạo xong trước khi chương trình start up

```
public class EagerInitialization {  
    private static final EagerInitialization INSTANCE = new EagerInitialization();  
    private EagerInitialization()  
    {  
    }  
    public static EagerInitialization getInstance()  
    {  
        return INSTANCE;  
    }  
}
```

Hiện thực/cài đặt

- Static block initialization

```
public class StaticInit {  
    private static StaticInit instance;  
    static {  
        try{  
            instance = new StaticInit();  
        }  
        catch (Exception e)  
        {  
            // xử lý exception bắt được  
        }  
        finally {  
            }  
    }  
    private StaticInit() {  
    }  
    public static StaticInit getInstance() {  
        return instance;  
    }  
}
```

Tương tự với phương pháp Eager, cách này cũng khởi tạo thể hiện của lớp singleton trước khi sử dụng, chỉ khác là nó cho phép ta có thể bắt và xử lý được exception nếu có trong quá trình khởi tạo.

Hiện thực/cài đặt

- Lazy Initialization

```
public class LazyInit {  
    private static volatile LazyInit instance;  
    private LazyInit() {  
    }  
    public static LazyInit getInstance()  
    {  
        if (instance == null)  
        {  
            instance = new LazyInit();  
        }  
        return instance;  
    }  
}
```

Cách này nó sẽ tạo thể hiện ở trong phương thức trả về. Trong lượt gọi đầu tiên, nó sẽ kiểm tra rằng thể hiện đã được khởi tạo hay chưa, nếu chưa thì nó sẽ tạo mới và trả về thể hiện đó, còn không thì nó sẽ trả về thể hiện đã được khởi tạo.

Hiện thực/cài đặt

- Thread Safe Singleton

```
public class ThreadSafeSingleton {  
    private static volatile ThreadSafeSingleton instance;  
    private ThreadSafeSingleton() {  
    }  
    public static synchronized ThreadSafeSingleton getInstance()  
    {  
        if (instance == null) {  
            instance = new ThreadSafeSingleton();  
        }  
        return instance;  
    }  
}
```

Một số so sánh

SINGLETON

- Khởi tạo instance khi cần sử dụng.
- Cố định một cách gọi instance.
- Hỗ trợ kế thừa.
- Lưu trữ trong Heap.
- Có thể clone Constructor được.
- Có thể triển khai từ Interface

GLOBAL VARIABLES

- Khởi tạo ngay từ ban đầu ngay cả không sử dụng
- Có thể định danh object cùng tên trong scope nhỏ hơn

STATIC CLASS

- Không hỗ trợ kế thừa.
- Lưu trữ trong Stack
- Không clone Constructor được.
- Không thể triển khai từ Interface

Một số ví dụ thực tế

- Singleton trong phát triển game



Một số ví dụ thực tế

- Singleton trong phát triển game

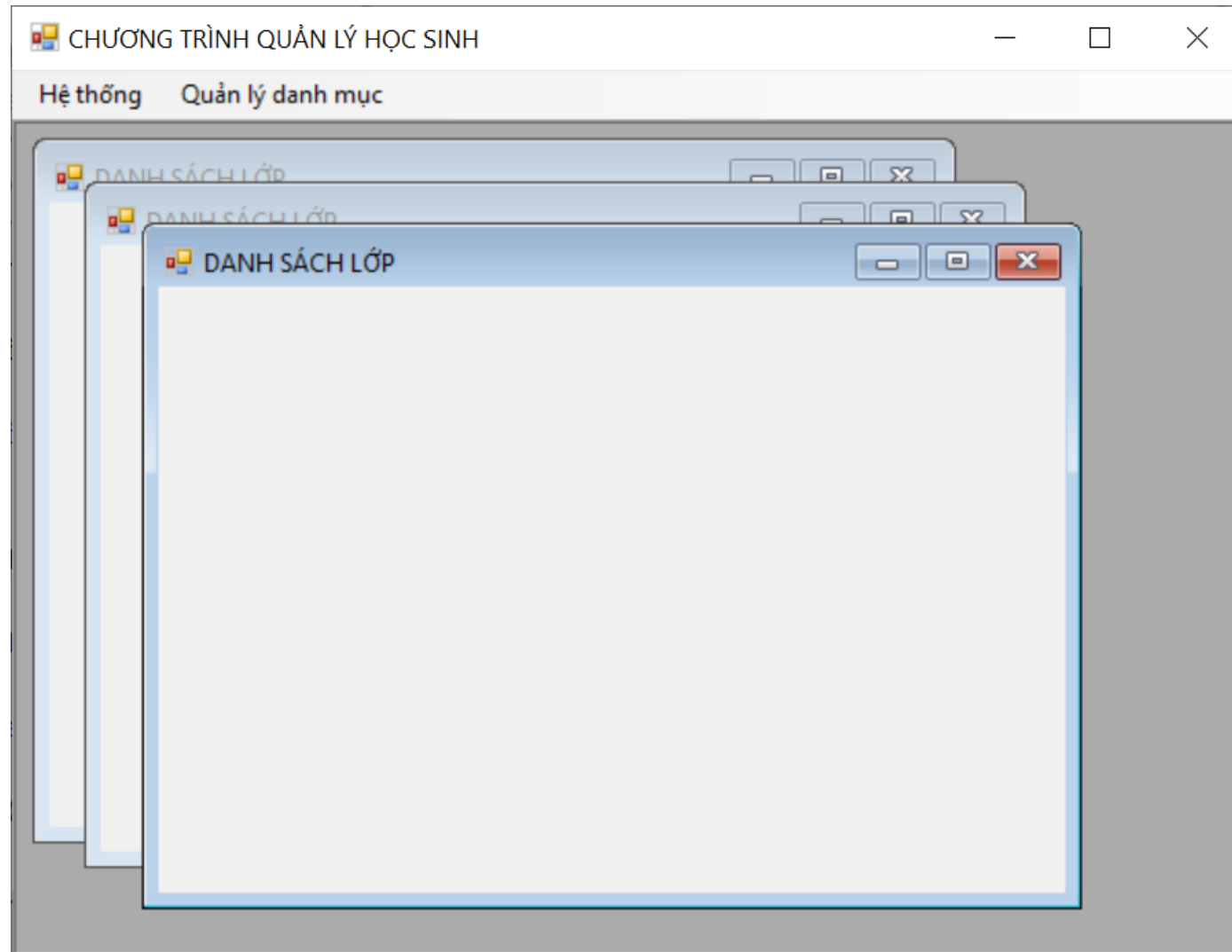
```
class Sound
{
private:
    Sound(HWND hWnd);
    static Sound* instance;
    IDirectSound8* pDevice;
    IDirectSoundBuffer* primaryBuffer;
    std::map<std::string, IDirectSoundBuffer8*> soundBufferMap;
```

```
Sound* Sound::getInstance()
{
    if (instance == nullptr) {
        instance = new Sound(CGame::GetInstance()->getCurrentHWND());
        instance->loadGameSounds();
    }

    return instance;
}
```

Một số ví dụ thực tế

- Mở Form mới trong ứng dụng window forms

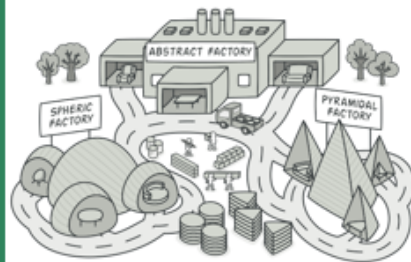


Các mẫu liên quan

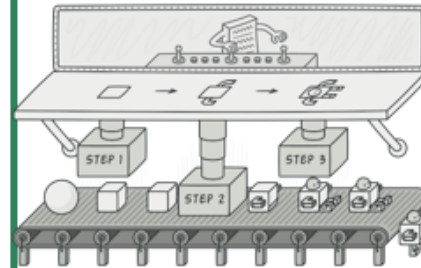
- Có rất nhiều mẫu có phần cài đặt, ý tưởng có sự liên quan nhất định đến Singleton, chẳng hạn như:



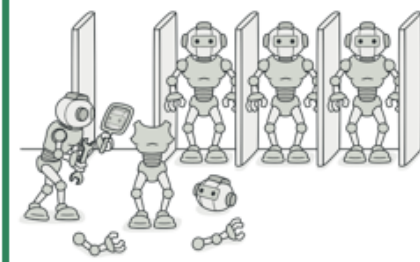
Abstract Factory



Builder



Prototype



Q & A

